

分类号 TP302

学号

UDC

密级 公开

工学博士学位论文

基于先验模型与深度学习的图像去噪
方法研究

博士生姓名

学科专业 控制科学与工程

研究方向 多媒体信息系统与虚拟现实技术

指导教师

二〇二六年三月

Joint Demosaicing and Denoising of Bayer Images

Candidate:

Supervisor:

A dissertation

Submitted in partial fulfillment of the requirements

for the degree of Ph.D of Engineering

in Control Science and Engineering

March, 2026

目 录

摘 要	i
Abstract	ii
第一章 绪论	1
1.1 研究背景	1
1.1.1 训练多模态大语言模型面临的挑战	1
1.1.2 异构 GPU 集群下的并行训练挑战	4
1.2 国内外研究现状	7
1.2.1 多模态大语言模型并行训练策略研究现状	7
1.2.2 异构 GPU 集群并行训练策略研究现状	8
1.3 研究内容与贡献	10
1.3.1 MMoH: 多模态大语言模型在异构 GPU 集群上的混合并行 ..	10
1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算	11
1.4 论文组织结构	11
第二章 相关工作	12
2.1 多模态大语言模型	12
2.1.1 多模态大语言模型的模型架构	12
2.1.2 多模态大语言模型的训练数据	13
2.1.3 多模态大语言模型的训练过程	15
2.2 并行与分布式训练技术	16
2.2.1 数据并行与模型并行	16
2.2.2 零冗余优化器 (ZeRO)	18
2.3 异构集群上的大模型并行训练	19
2.3.1 异构 GPU 集群下的并行训练策略	20
2.3.2 异构 GPU 集群并行训练的工业实践	21
2.4 重计算显存优化技术	22
2.4.1 重计算基本原理与分类	22
2.4.2 重计算的主流实现及其局限性	24
2.5 本章小结	24
致谢	26
参考文献	27

作者在学期期间取得的学术成果	27
附录 A 本文中英文对照表	28

表 目 录

表 2.1	多模态大语言模型训练中常用的部分数据集规模示意	14
表 A.1	本文主要术语中英文对照表	28

图 目 录

图 1.1	多模态大语言模型的核心组成部分	3
图 1.2	NVIDIA GPU 架构与代表性产品演进	5
图 1.3	使用 Megatron 训练多模态大语言模型	9
图 2.1	MLLM 多阶段训练中典型模块冻结与更新关系示意	16
图 2.2	数据并行示意	17
图 2.3	张量并行示意	17
图 2.4	流水线并行示意	18
图 2.5	专家并行（EP）中门控路由与专家分布示意	19
图 2.6	零冗余优化器 ZeRO 示意	19
图 2.7	重计算（激活检查点）示意	22

摘 要

多模态大语言模型（MLLM）融合视觉与语言等多模态能力，训练过程高度依赖分布式并行计算。面向真实场景时，GPU 集群通常由多代异构设备组成，显存、算力的代际差异明显。与此同时，MLLM 由编码器、投影层与 LLM 基座等异构子模块构成，并普遍采用分阶段冻结训练，导致不同训练阶段的计算负载与显存需求动态变化。现有并行训练方法多基于同构集群与统一配置的前提，难以同时解决异构 GPU 环境下的负载均衡、冻结阶段冗余计算与通信、以及显存与效率协同优化的问题。

针对异构集群与分阶段冻结训练下并行策略难以自适应、易出现负载不均衡与冗余反向通信的问题，本文提出研究方法一：**MMoH** 框架。该方法通过需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知整数规划划分，自动生成面向不同训练阶段的混合并行策略，并通过提前终止调度机制裁剪连续冻结流水级中的冗余反向计算与流水级间通信。在两种异构集群与两种规模 MLLM（3B、12B）上的实验表明，MMoH 在多种冻结场景下相较 Megatron-LM、Whale 与 Espresso 均取得最高吞吐，最高加速约 1.77 倍，且提前终止调度可额外带来约 8%–19% 性能增益。

针对现有全局统一重计算策略在异构集群上难以兼顾显存安全与计算效率的问题，本文在 MMoH 框架的基础上提出研究方法二：**HR-MMoH** 异构重计算方法。该方法在 MMoH 划分与调度结果基础上，按流水级进行差异化重计算配置，并结合剖析驱动的配置搜索形成“划分—调度—重计算”的协同优化方法。在增大的批次大小与序列长度的端到端实验中，HR-MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种场景下均取得最高最大吞吐，迭代时间相对 Espresso 缩短约 6.1%–23.1%，相对 Whale 的可训练子集缩短约 11.0%–40.6%，验证了该方法在异构 GPU 集群上的有效性与可推广性。

关键词: 多模态大语言模型；异构 GPU 集群；流水线并行；重计算

Abstract

Multimodal large language models (MLLMs) integrate visual and linguistic capabilities, and their training heavily depends on distributed parallel computing. In real-world deployments, GPU clusters are usually heterogeneous across generations and device types, with significant differences in memory capacity, compute capability, and communication bandwidth. Meanwhile, MLLMs are composed of heterogeneous modules (encoders, projectors, and LLM backbones) and are commonly trained with stage-wise freezing, which introduces dynamic shifts in compute and memory demands. Existing parallel training methods are mainly designed under homogeneous-cluster and uniform-configuration assumptions, and thus struggle to jointly address load imbalance, redundant computation/communication under freezing, and memory-efficiency trade-offs in heterogeneous environments.

To address the lack of adaptive parallel strategies under heterogeneous clusters and stage-wise freezing, where load imbalance and redundant backward communication are common, this thesis proposes Method I: MMoH. MMoH combines demand-driven device-topology identification, compound-granularity model abstraction, and freeze-aware integer-programming partitioning to automatically generate stage-matched pipeline strategies, and introduces an early-termination scheduler to prune redundant backward computation and inter-stage communication in consecutive freezing stages. Experiments on two heterogeneous clusters and two model scales (3B and 12B) show that MMoH consistently achieves the highest throughput over Megatron-LM, Whale, and Espresso across multiple freezing settings, with up to $1.77\times$ speedup and an additional 8%–19% gain from the early-termination scheduler.

To address the limitation of globally uniform recomputation, which cannot jointly balance memory safety and computational efficiency on heterogeneous clusters, this thesis proposes Method II: HR-MMoH for heterogeneous recomputation. Built on top of MMoH partitioning and scheduling outputs, HR-MMoH applies stage-wise differentiated recomputation configurations and couples them with profiling-guided configuration search to form a joint optimization pipeline over partitioning, scheduling, and recomputation. In end-to-end experiments with enlarged batch size and sequence length, HR-MMoH attains the highest maximum throughput in all four settings (no-freeze, freeze-encoder,

freeze-llm, and freeze-both), reducing iteration time by 6.1%–23.1% versus Espresso and 11.0%–40.6% versus Whale (trainable subset), validating its effectiveness and practical scalability on heterogeneous GPU clusters.

Key Words: Multimodal large language model; Heterogeneous GPU cluster; Pipeline parallelism; Recomputation

本文主要符号使用说明

HPC	高性能计算 (High Performance Computing)
cluster	集群
Itanium	安腾
SMP	对称多处理
API	应用程序编程接口
PI	聚酰亚胺
MPI	聚酰亚胺模型化合物, N- 苯基邻苯酰亚胺
PBI	聚苯并咪唑
MPBI	聚苯并咪唑模型化合物, N- 苯基苯并咪唑
PY	聚吡咙
PMDA-BDA	均苯四酸二酐与联苯四胺合成的聚吡咙薄膜
ΔG	活化自由能 (Activation Free Energy)
χ	传输系数 (Transmission Coefficient)
E	能量
m	质量
c	光速
P	概率
T	时间
v	速度

第一章 绪论

1.1 研究背景

1.1.1 训练多模态大语言模型面临的挑战

深度神经网络（Deep Neural Network, DNN）在计算机视觉（Computer Vision, CV）、自然语言处理（Natural Language Processing, NLP）、语音识别（Speech Recognition, SR）等众多前沿领域取得了优异的效果，近年来逐步演变为诸多关键前沿技术领域的核心基础 [?]。2017 年，Vaswani 等人提出基于自注意力机制（Self-Attention）的 Transformer 架构 [?]，摒弃了传统的循环神经网络（Recurrent Neural Network, RNN）与卷积神经网络（Convolutional Neural Network, CNN）结构，由于可以并行计算而且对于长距离信息捕捉能力强，Transformer 架构迅速成为自然语言处理与跨模态建模的主流模型结构。

在自然语言处理方面，基于 Transformer 的预训练大语言模型（Large Language Models, LLMs）不断刷新在何种基准测试成绩。BERT[?] 通过双向编码与掩码预训练在语言理解类任务上表现突出；GPT-2[?]、T5[?] 等自回归（Autoregressive）或编码器 - 解码器（Encoder-Decoder）模型则在文本生成上有优势；GPT-3[?] 与后续的 GPT-4[?] 等大规模语言模型更在少样本（Few-Shot）与零样本（Zero-Shot）设定下展现出接近人类的表达能力。在计算机视觉领域，Vision Transformer（ViT）[?] 及后续的规模化工作 [?] 表明：将图像切分为块（patch）并经 Transformer 编码，即可在 ImageNet 等基准数据集上超越传统卷积神经网络。这表明 Transformer 架构的模型不仅可以处理文本，还具有处理视觉等其余模态信息的潜力。

随着单模态模型已经接近甚至超越人类水平，将语言、视觉、音频、表格等多模态信息纳入统一的模型框架进行联合表示与推理，成为当前学术界和工业界研究的新方向。多模态（Multimodality）是指可以同时处理两种或两种以上模态信息的能力。在机器学习与人工智能中，常见模态包括文本、图像、视频、音频、表格以及各类传感器数据，更贴近于人类接触到的复杂信息。多模态大语言模型（Multimodal Large Language Models, MLLMs）特指在大语言模型的基础上，融合对视觉、听觉等其它模态的感知与生成能力的一类模型 [???]。MLLMs 的先驱工作 CLIP[?]（Contrastive Language-Image Pre-Training）通过使用图像文本对进行训练，使得模型拥有理解图片的能力，DALL-E[?] 则与 CLIP 相反，展示了从文本生成图像的可行性。近年来，MLLMs 在架构与训练范式上发展迅速：BLIP-2[?] 提出 Q-Former 连接冻结的视觉编码器与 LLM 进行训练；InstructBLIP[?] 在此基

础上加强了视觉信息的提取能力，在多项零样本任务上取得 SOTA 结果；LLaVA[?] 与 LLaVA-1.5[?] 采用简单线性投影层实现视觉与语言模态对齐，验证了“小连接器 + 大模型”的有效性；Qwen-VL[?]、InternVL[?] 等系列多模态模型则进一步在长上下文、高分辨率与多图理解上扩展模型能力。这类模型既保留 LLMs 在语言理解、推理与生成上的优势，又能够直接处理图像、视频等多模态输入，实现跨模态交互与推理任务，因而被认为是迈向通用人工智能（Artificial General Intelligence, AGI）的潜在路径[?]。多模态大语言模型能力的拓展既依赖参数与数据规模的提升，也依赖编码器、投影层、基座模型与可选生成器等模块的协同设计：前者关系到模型可达的效果上限，后者则直接决定训练阶段中计算与显存在各模块间的分布形态。

从功能表现来看，MLLMs 可完成图像描述、视觉问答、图文对话、文档理解，以及基于文本的图像或视频生成等复杂任务，在诸多前沿领域比如自动驾驶、智能助手等场景具有广阔应用前景；在垂直领域，MLLMs 已延伸至具身智能[?]、医疗影像理解[?]等。多模态大模型的相关评估常使用 VQA_{v2}[?]、GQA[?]、MMMU[?] 等多模态基准，对模型规模与训练数据提出了更高要求。从结构上看，MLLMs 通常包含以下核心组件，如图1.1：（1）*Modality Encoder*，负责将图像、视频等非文本输入编码为特征表示，常采用预训练的视觉模型（如 CLIP、ViT）作为 backbone；（2）*Input Projector*，将编码后的多模态特征映射到与 LLM 词嵌入对齐的语义空间，常见形式包括线性层、Q-Former[?] 等轻量模块；（3）*LLM Backbone*，即大规模语言模型主体，承担主要的理解与生成计算，参数量通常占整体模型的 70%–80%；（4）*Output Projector*（在需要多模态生成时），将 LLM 输出映射到与目标模态生成器对齐的表示空间；（5）*Modality Generator*（在需要多模态生成时），负责生成图像、音频等目标模态，常采用扩散模型（如 Stable Diffusion）等。其中 LLMs 处于 MLLMs 架构的核心，多模态能力是在其基础上进行的扩展与增强。

在大语言模型占据主体参数量的架构下，针对 LLMs 总结出的扩展定律（Scaling Laws）与数据规模规律，同样深刻影响着 MLLMs 训练的资源需求。已有实证研究表明，模型性能通常会随参数规模与训练数据规模的增加而稳定提升，这一现象被总结为“扩展定律”（Scaling Laws）[?]。Kaplan 等[?] 的早期工作表明，在给定算力预算下，模型规模与数据规模应按幂律关系共同扩展。Hoffmann 等[?] 进一步指出，当前大语言模型普遍“训练不足”，在计算最优意义下，模型参数量与训练 token 数应近似同比例扩展（约 20 倍 token/ 参数），该结论对预训练数据规模与模型架构选择产生了深远影响。近期工作[?] 则从推理成本角度拓展了 Chinchilla 最优，指出在考虑部署与推理时，最优扩展策略会随推理请求规模发生变化。为追求更强性能，工业界与学术界不断推出参数量更大、数据规

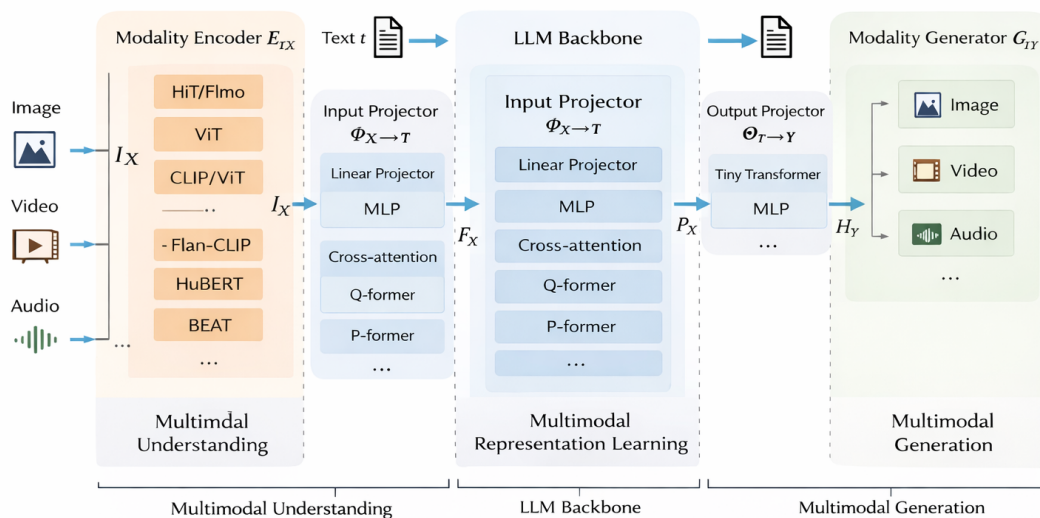


图 1.1 多模态大语言模型的核心组成部分

模更大的模型，从数亿参数（如 BERT）到千亿、万亿参数（如 GPT-3、Gopher[?]]、DeepSeek-V3[?]]、Qwen2.5[?]]、LLaMA 2/3[?] ?]、Gemma[?]]、Mistral[?]] 等开源与闭源 LLMs），单卡或单机 GPU 的显存与算力已无法容纳整模型的完整训练流程 [?] ?]。相应地，多模态大模型的预训练与指令微调所需的数据量也急剧增长，从数百万图文对到数十亿级样本，单机算力、存储与 I/O 同样成为瓶颈；数据质量与混合比例对下游性能同样有显著影响 [?]]，长上下文与多模态联合基准 [?]] 的普及进一步推高了对训练规模与效率的需求。在上述模型与数据规模的约束下，单台计算设备的显存与算力均难以独立支撑完整的训练流程。因此必须借助分布式与并行训练技术，将模型参数、优化器状态、激活值与数据分散至多台设备，通过节点间的通信协同完成大规模模型的训练 [?] ?]。

目前，支撑大规模 MLLMs 训练的主流并行策略可归纳为两大类。第一类是以计算与模型切分为核心的多维混合并行：数据并行 [?] ?] 在各设备上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且在小模型上扩展性好；张量并行 [?] ?] 将单层内的矩阵按行或列切分到多卡，降低单卡显存并保留层内并行，是千亿级稠密模型的主流选择；流水线并行 [?] ?] 将模型按层划分为若干流水级并分配到不同设备，通过微批次与调度重叠计算与通信，缓解单设备层数过多的问题；序列并行 [?]] 与专家并行 [?]] 则分别在长序列注意力与混合专家（MoE）结构中沿序列维或专家维切分，与张量、流水线并行组合后可进一

步扩展模型规模。第二类是以显存优化为核心的零冗余优化器系列：ZeRO[?] 将优化器状态、梯度与参数分片存于各卡，前向与反向时按需聚合，在保持数据并行通信量的同时显著降低单卡显存；ZeRO-Infinity[?] 进一步将部分状态卸载至 CPU 甚至是 NVMe 上，突破 GPU 显存墙；ZeRO++[?] 通过量化与通信重算技术减少跨节点通信量，提升多节点训练效率。上述并行策略常与混合精度训练[?]、重计算[?]、Flash Attention[?] 等显存与计算优化技术结合，共同构成当前大模型与 MLLMs 训练的基础设施；PyTorch 的 FSDP[?] 与 DeepSpeed ZeRO 类似，通过全分片参数与优化器状态实现单卡显存的大幅降低。

然而，上述策略在设计时多假设同构集群与单一形态的 Transformer 模型，其划分粒度、负载均衡与通信优化均围绕“各层计算与显存相对均匀”这一前提展开；在面对异构 GPU 集群与模块异构的 MLLMs 时，负载与显存分布的不均匀性被放大，仍存在效率与可扩展性方面的挑战。在实际应用中，企业与研究机构常面临定制化多模态模型的高频微调需求，这要求训练系统具备较高的可扩展性与资源利用率。然而，受预算与硬件采购周期的限制，多数集群难以维持绝对的同构性，往往需要在现有的异构设备组合上开展预训练与微调实验。因此，提升异构 GPU 集群下的训练效率、使其逼近同构环境的表现，具有迫切的现实需求。同时，因负载不均或通信瓶颈导致的计算资源浪费不仅会延长训练周期，亦会显著增加算力成本。由此可见，针对异构集群设计高效的 MLLMs 并行训练方案，既是系统层面的技术挑战，也是落地部署中的关键环节。

1.1.2 异构 GPU 集群下的并行训练挑战

多模态大语言模型的训练效率与成本高度依赖底层硬件平台。目前，工业界与学术界普遍采用 NVIDIA GPU 作为 MLLMs 训练的主要加速器，并通过多卡、多机集群进行分布式训练。然而，NVIDIA 约每 1-2 年迭代一代计算架构并推出多款不同定位的 GPU，导致显存容量、算力与互联方式持续分化，实际部署中的集群往往由多代、多型号 GPU 混合组成，形成显存异构、算力异构与通信异构并存的局面[?]。对单层计算与显存相对均匀的传统 LLMs，尚可通过“按层数或算力比例”划分在一定程度上缓解异构；而对编码器、投影层与 LLMs 在结构与参数量上差异显著的 MLLMs，模块间负载差异与设备能力差异叠加，使得简单比例划分难以同时兼顾负载均衡与显存安全。

图1.2 概括了近年来 NVIDIA GPU 的架构演进。2016–2017 年的 Pascal（如 P100）与 Volta（V100）奠定了数据中心 GPU 与 Tensor Core 的基础；2018 年 Turing（RTX 20 系列）引入消费级光追与 INT8 推理；2020 年 Ampere 架构推出 A100（40GB/80GB HBM2e）与消费级 RTX 30 系列（如 RTX 3090 24GB），支持

PCIe 4.0 与 NVLink 3.0，成为当前许多实验室与云平台的主力 [?]。2022 年 Hopper 架构的 H100（80GB/96GB HBM3）针对大语言模型训练做了专门优化 [?]，并广泛配备 NVLink 4.0 与 NVSwitch，单机多卡带宽可达数百 GB/s；同年 Ada Lovelace 架构的 RTX 40 系列（如 RTX 4090 24GB）在消费级市场提供高性价比算力。2024 年起，Blackwell 架构的 B100/B200 等数据中心 GPU 逐步量产，采用 HBM3e、更高带宽的 NVLink5 与 PCIe 6.0，进一步拉大与旧代设备的性能与互联差距 [?]。按 NVIDIA 已披露的路线图，后续还将推出 Rubin（约 2026 年）、Rubin Ultra（约 2027 年），以及面向推理与能效深度优化的 Feynman 架构（约 2028 年），后者将采用下一代 HBM 与 3D 堆叠内存等技术，进一步凸显代际间显存与互联的差异。与此同时，不同代际与型号在 PCIe 版本（3.0/4.0/5.0/6.0）、是否配备 NVLink/NVSwitch、以及 InfiniBand 与 RoCE 等节点间网络上的差异，使得通信带宽与延迟在集群内也存在异构性。

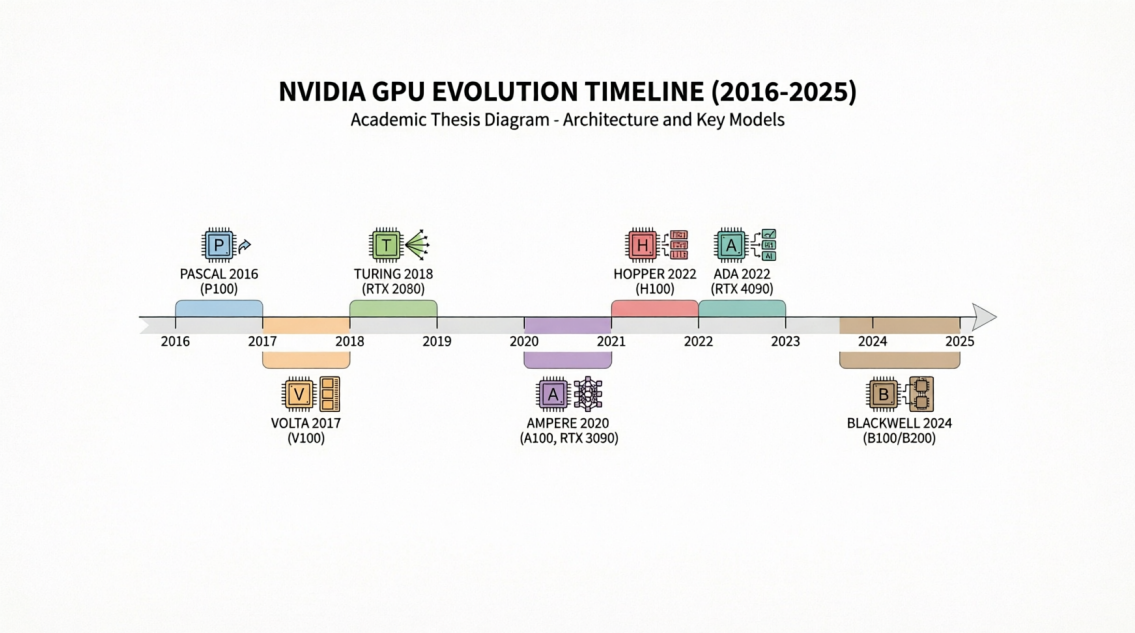


图 1.2 NVIDIA GPU 架构与代表性产品演进

在大多数高校、研究所与中小企业的实际环境中，受预算与采购周期限制，集群常同时包含多代 GPU（例如部分 A100 或 H100 与大量 RTX 3090/4090、甚至更早的 V100/RTX 2080），单机内也可能混用不同显存与互联配置，难以做到“全集群同构”。而主流并行训练框架（如 Megatron-LM、DeepSpeed）在设计时多假设同构设备与统一拓扑 [?]，在异构集群上直接使用往往导致：负载无法按算力与显存合理分配、通信成为瓶颈或部分高端卡空闲、以及因“木桶效应”整体效率下降 [?]。因此，在显存、算力与通信三重异构的 GPU 集群上高效训练 MLLMs，

是学术界和工业界共同面临且亟待解决的问题。

从系统层面看，异构带来的问题不仅在于单卡能力差异，还在于拓扑与调度：节点内 NVLink 与节点间 InfiniBand 的带宽可相差一个数量级以上，若流水线或张量并行的划分未考虑链路差异，跨节点通信极易成为瓶颈；同时，MLLMs 各阶段对编码器、投影层与 LLMs 的冻结策略不同，导致同一模型在不同训练阶段下各层的计算与显存占比发生明显变化，若沿用固定划分与调度，难以在整轮训练中保持负载均衡。因此，面向异构集群的 MLLMs 训练需要同时考虑硬件拓扑、模型结构与多训练阶段，通过自适应的划分与高效的调度使各设备利用率最大化。

除设备侧算力、显存与通信差异外，模型结构异构也会显著改变并行划分与负载形态。典型 MLLM 中，*Modality Encoder* 多采用视觉主干（如 CLIP、ViT 等），将图像或视频编码为特征；*LLMs Backbone* 常为 GPT 类或 Llama 类等大语言模型 [?]，参数量往往占整模型的约 70%–80%；可选的 *Modality Generator* 则可能是扩散模型（如 Stable Diffusion[?]）等，用于生成图像、视频或音频。LLM 部分通常由大量同构 Transformer 层堆叠；Encoder 与 Generator 往往融合卷积、U-Net[?] 或扩散去噪网络等不同算子；Input/Output Projector 则可能是线性层、小型 Transformer 或 Q-Former[?] 等轻量连接器。与“单层形态相对一致、逐层堆叠”的传统稠密 LLM 相比，MLLM 各模块在算子类型、参数量与激活体积上差异显著，同一划分策略在不同 GPU 上呈现的时间与显存开销不再近似齐次；在异构集群上，设备差异与模块差异叠加，负载不均与显存风险更易被放大。

多模态大语言模型的训练流程也与纯文本 LLM 常见做法不同。得益于视觉、语言与生成等领域成熟的预训练成果，当前主流范式大量复用已有强模型，并在训练中通过冻结与解冻控制可训参数规模。BLIP-2[?] 较早系统性地采用“冻结视觉编码器与 LLM、主要训练 Q-Former 等连接器”的策略完成跨模态对齐。此后，这种基于冻结与局部更新的策略被广泛应用于各类多模态大语言模型中，如近期的 NextGPT[?]、DreamLLM[?] 等工作。此外，多模态大语言模型的训练通常被划分为多个阶段（如模态对齐、指令微调等），不同阶段针对特定能力解冻不同模块。当某一模块被冻结时，其反向传播的计算量会大幅下降；且优化器亦无需为冻结参数维护一阶与二阶矩等状态变量（如使用 Adam[?] 优化器时），这显著降低了训练过程中的显存压力。但在异构 GPU 集群中，这种由模块冻结带来的算力、显存与通信需求的时变性，反而放大了各设备间的负载不均。此外，云上与混合部署中的弹性训练与按需资源供给，使得集群内设备型号与拓扑更加多样，对异构感知的划分与调度、以及训练成本与性能的联合优化提出了进一步要求 [??]。

硬件层面的设备异构与模型层面的模块异构、分阶段冻结交织，构成了当前

有限资源下高效训练的核心难点。为此，本课题拟结合现有并行策略，深入分析实际 GPU 集群中的异构场景，基于 Megatron 等主流并行训练框架，针对 MLLMs 的结构特点以及集群内显存、算力与带宽的异构性，研究细粒度的混合并行技术和重计算显存优化技术，旨在提出一套面向异构复杂环境的自适应并行训练方案来缓解异构集群下训练 MLLMs 效率的问题。

1.2 国内外研究现状

本节从文献角度归纳与本文密切相关的研究脉络：先概述 MLLM 的规模演进、数据需求与主流并行技术栈及其同构假设局限，再综述面向异构 GPU 集群的并行与调度工作，并与绪论中的问题动机相呼应。

1.2.1 多模态大语言模型并行训练策略研究现状

近年来，基座语言模型、视觉模型与多模态大语言模型持续向更大参数规模与更大数据规模演进。Kaplan 等 [?] 通过大量实验归纳出扩展定律（Scaling Law）：模型性能随模型参数量与训练数据量的幂律关系持续提升，这为“做大模型、用大数据”提供了理论依据；Chinchilla[?] 进一步表明，在计算最优意义下，模型规模与训练 token 数应同比例扩展，后续研究 [?] 从推理成本角度拓展了最优缩放策略。提升大模型性能的两条主要途径是增大模型参数与扩大数据规模 [?]：通过提升参数量可增强对复杂场景的拟合与泛化能力；通过扩大数据规模可使模型学习到更充分的特征表示，更好地适应复杂应用场景；参数高效微调（如 LoRA[?]、QLoRA[?]）则在资源受限下广泛用于 MLLM 的适配与部署 [?]

自 Transformer[?] 提出以来，语言模型规模迅猛增长：从 BERT[?] 的亿级参数，到 GPT-2[?]、T5[?] 的十亿级，再到 GPT-3[?] 的千亿级与 GPT-4[?] 等闭源模型，以及 DeepSeek-V3[?]、Qwen2.5[?] 等开源千亿级模型。多模态大语言模型在此基础上融合视觉编码器与语言模型，代表性工作包括 BLIP-2[?]（Q-Former 连接视觉与语言）、InstructBLIP[?]（指令感知视觉特征）、LLaVA/LLaVA-1.5[?]（线性投影对齐）、Flamingo[?]（交错图文与 gated 交叉注意力）、Qwen-VL[?]、InternVL[?]、MiniGPT-4[?] 等，其参数量从数十亿到数百亿不等，且训练数据从数百万图文对到数十亿级样本（如 LAION-5B[?]）；相关综述从架构、对齐策略、数据与效率等角度对 MLLMs 进行了系统梳理 [?]。单卡或单机在显存与算力上已无法容纳完整模型与训练流程 [?]，并行与分布式训练成为训练大模型与 MLLM 的必由之路 [?]

训练数据规模同样持续扩大：图像方面，Open Images[?]、COCO[?] 等需 TB 级存储；LAION-5B 等开放图文对达数十亿样本；文本方面，Common Crawl、

The Pile[?] 等为 LLM 预训练提供数百 GB 至数万亿 token 量级语料；视频方面，YouTube-8M[?]、WebVid[?] 等基准需 PB 级存储与高吞吐数据管线；多模态理解与推理则依赖 VQAv2[?]、TextVQA[?] 等数据集与长上下文基准[?]。在单机无法存储完整数据集与模型的前提下，并行训练（数据并行、模型并行及其组合）成为必然选择。

在并行策略方面，工业界与学术界已形成以数据并行、张量并行、流水线并行及 ZeRO 等为核心的技术栈，并被 Megatron-LM、DeepSpeed 等框架广泛应用于千亿级参数模型训练[?]。Galvatron[?] 针对 Transformer 在多 GPU 上的自动并行进行了系统探索，将策略搜索与编译优化结合；Colossal-Auto[?] 等将自动并行策略搜索与重计算统一自动化，为大规模与异构环境下的联合优化提供了参考。然而，上述框架在设计时多假设同构 GPU 集群与单一形态的 Transformer 模型，对 MLLMs 的模块异构（编码器、投影层、LLM、生成器结构各异）与分阶段冻结考虑不足；在实际部署中，集群往往由多代、多型号 GPU 混合组成，存在显存、算力与通信的异构[?]，直接使用传统并行策略易出现负载不均、通信瓶颈与“木桶效应”。因此，面向异构 GPU 环境设计多模态大模型并行训练方法与重计算策略，实现自感知、自适应、细粒度的划分与调度，对提高算力利用率、充分利用显存空间十分必要。而要支撑上述目标，需依托现有并行训练的主体形态，并在此基础上针对异构集群与 MLLM 做专门适配。

1.2.2 异构 GPU 集群并行训练策略研究现状

并行训练是当前工业界与学术界训练多模态大语言模型的主要手段，主流做法可概括为两类：以计算与张量切分为核心的多维混合并行，以及以显存分片为核心的零冗余优化器（ZeRO）系列。

多维混合并行通过在不同维度上划分模型或数据以降低单机负载。数据并行[?] 在各设备上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且扩展性好。张量并行[?]（如 Megatron-LM）将单层内矩阵按行或列切分到多卡，降低单卡显存，是千亿级稠密模型的主流选择。流水线并行将模型按层划分为若干阶段并分配到不同设备：GPipe[?] 通过微批次与多阶段调度减少流水线气泡；PipeDream[?] 提出 1F1B 等调度进一步压缩空闲时间。此外，序列并行[?]、专家并行[?] 等与张量、流水线并行组合后可进一步扩展模型规模。NVIDIA 的 Megatron-LM[?] 联合使用张量、流水线与数据并行，已支撑在数千张 GPU 上训练千亿至万亿级参数的大语言模型；Korthikanti 等[?] 针对超大 Transformer 提出序列并行与选择性重计算，与张量并行协同以降低激活显存。零冗余优化器 ZeRO[?] 在数据并行基础上将优化器状态、梯度与参数分片

存于各卡、按需聚合，显著降低单卡显存；ZeRO-Infinity[?] 将部分状态卸载至 CPU/NVMe；ZeRO++[?] 通过量化与通信重算减少跨节点通信量。

然而，上述策略多假设同构设备与统一拓扑。Megatron-LM 在设计时未考虑设备异构与模型异构；在实际异构环境下训练 MLLMs 时，往往将不同型号、显存与带宽的 GPU 当作同构设备统一配置，其典型并行策略如图1.3所示。对于 MLLM 中结构各异的 Modality Encoder、LLM Backbone、Projector 与 Modality Generator，Megatron 缺乏针对性的细粒度划分与映射，编码器与生成器的并行方式通常被动跟随 LLM 主干划分，难以根据各子模块的算力与显存需求及不同 GPU 能力做自适应分配，导致显存小、算力弱或通信慢的 GPU 成为瓶颈，整体利用率与训练效率下降 [? ?]。

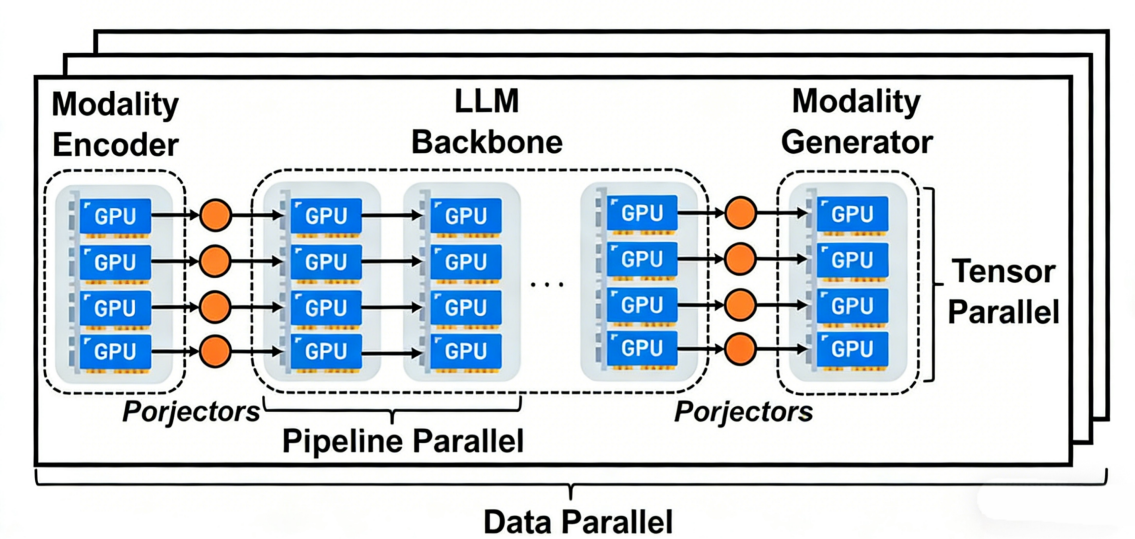


图 1.3 使用 Megatron 训练多模态大语言模型

针对异构 GPU 集群的并行训练，已有不少研究。HetPipe[?] 较早将流水线并行与数据并行结合，通过虚拟工作节点（VW）在异构设备间分配负载，但未显式建模设备排列与通信拓扑，也未考虑 MLLM 的模块异构与冻结策略。Whale[?] 针对显存与算力异构提出基于硬件感知的负载均衡方法，按比例对各设备分配不同大小的模型分片与批次，但未考虑网络异构，划分粒度相对粗糙。AMP[?] 在异构环境下为大模型自动搜索并行策略，同时考虑模型异构与设备异构，通过开销模型与动态规划求解近似最优配置，但划分粒度为全局粗粒度，对通信与显存估计的准确性有限。近年来，面向异构集群的分布式训练系统与调度方法受到持续关注。HetHub[?] 支持 AMD、NVIDIA 等多厂商异构 GPU 集群，将通信器、性能预测器与自动并行统一规划；HexiScale[?] 将数据、流水线与张量并行的非对称划分形式化，采用层次图划分算法在异构环境下分配计算；Espresso[?] 面向成本

效益的异构 GPU 资源供给与阶段放置，集成 Profiler、GPU Allocator、Stage Placer 与 Performance Estimator；上述工作多为通用 LLM 或同构扩展设计，对 MLLM 的模块异构、分阶段冻结与显存—算力—通信三重异构的联合优化仍不足。

ZeRO[?] 的梯度 / 参数 All-Gather 等通信在跨节点异构网络中易成为瓶颈；MiCS[?] 对异构网络下的高带宽链路利用仍有不足；ZeRO++[?] 的量化与通信重算可能引入精度与通用性代价；AMSP[?] 等侧重跨节点通信削减，对集群内存储、算力与带宽的细粒度异构利用仍有限。因此，在异构 GPU 集群上高效训练 MLLMs，仍需在 DeepSpeed 与 Megatron 等现有框架基础上，引入对设备能力与模型结构的感知与自适应优化。

1.3 研究内容与贡献

1.3.1 MMoH：多模态大语言模型在异构 GPU 集群上的混合并行

针对现有并行策略在异构集群适配上的局限性，本课题以实际 GPU 集群中普遍存在的显存、算力和网络异构为切入点，研究细粒度的混合自动并行优化技术。提出适用于异构 GPU 环境的 MLLMs 自适应并行训练方案，以期在复杂集群下缓解负载不均与通信瓶颈，提升整体训练效率。

针对异构环境下显存利用不均、通信链路差异导致的带宽瓶颈、算力差异导致的设备利用率下降、模块结构差异导致的算力分配困难，以及分阶段冻结引起的计算与显存需求随时间变化等问题，本课题提出一种面向异构设备与异构模型结构的多模态大语言模型计算图层级划分方法，通过为不同属性的计算设备分配不同大小的部分模型，实现计算资源、存储空间、通信带宽的充分利用，争取最大限度提高训练速度。

MLLM 训练通常划分为多阶段（如对齐、指令微调等），各阶段对 Encoder、Projector、LLM 等模块采取不同冻结策略：冻结模块的反传与优化器状态维护显著减少，而未冻结模块仍承担主要计算与通信。阶段切换后，若仍沿用固定流水线划分与 1F1B 等常规调度，易出现阶段间耗时失衡、流水线气泡居高不下的现象，并可能伴随与冻结状态不相称的冗余反传与 P2P 通信。

针对上述问题，本课题提出面向多模态大语言模型的提前终止流水线调度策略。该策略感知各训练阶段的冻结格局，与划分方案协同调整；并在连续冻结的流水级上省略不必要的反向计算与阶段间通信，在保持梯度语义一致的前提下提高多卡流水线的有效利用率与训练吞吐。

1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算

重计算通过在前向时丢弃部分中间激活、在反向时按需重算，以计算换显存，是支撑大模型与 MLLMs 在有限显存下训练的重要手段。然而，现有重计算策略多采用全局统一的配置（如对全部层或按固定间隔插入检查点），未考虑异构集群中设备间显存与算力的差异：显存紧张、算力相对充裕的设备适合更激进的检查点以释放显存；显存宽裕、算力紧张或通信繁忙的设备则不宜过度重算，否则会加重计算或通信瓶颈。此外，MLLMs 各子模块（Encoder、LLM、Projector、Generator）的层数、每层激活体积与计算量各不相同，在同一设备上对不同模块采用相同检查点策略也难以达到最优。

本课题针对上述不足，提出一种面向异构设备与异构模型的重计算优化方法。该方法在给定层级划分与设备映射的基础上，以各设备的显存上限以及算力为约束，为不同设备上的不同模型阶段分别搜索或配置差异化的检查点策略，在满足显存不溢出的前提下，最小化因重计算带来的额外时间开销，并兼顾流水线不同流水级的负载均衡。通过将重计算与前述的复合粒度划分、流水线调度相结合，实现计算、存储与通信在异构环境下的联合优化，进一步提升 MLLMs 在异构 GPU 集群上的训练效率。

1.4 论文组织结构

本论文共分为五章，具体结构如下：

第一章为绪论。在规模扩展与并行训练需求、设备与拓扑异构、模型结构与分阶段冻结三个层面阐述研究背景与意义，归纳国内外相关研究现状，并概述本文的研究内容与核心贡献。

第二章为相关工作。从 MLLMs 架构特点、分布式训练基础、异构 GPU 并行策略以及重计算显存优化等方面梳理现有研究的发展脉络与局限性。

第三章介绍本文提出的 MMoH 框架，重点阐述需求驱动的设备拓扑识别、复合粒度模型抽象、冻结感知划分及提前终止流水线调度方法，并给出实验评估。

第四章介绍 HR-MMoH 异构重计算优化，详细说明如何在异构计算资源与不同模型模块间实现重计算策略的细粒度自适应配置并进行实验验证。

第五章为总结与展望。对全文工作进行回顾，并探讨后续可能的研究方向。

第二章 相关工作

本章对本文涉及的相关技术进行展开叙述，全章按以下顺序组织：(1) MLLMs 的架构模块、训练数据规模与多阶段冻结训练范式相关研究；(2) 数据、张量、流水线、序列与专家并行及 ZeRO 等主流并行训练方法；(3) 异构集群上的大模型并行训练与调度相关研究；(4) 重计算策略分类、框架实现与同构全局配置的局限。在此基础上收束到本文关注点：在异构设备 + 模块异构 + 分阶段冻结叠加时，现有同构导向划分、调度与统一重算策略仍难以同时保证负载均衡、显存安全与吞吐，从而为第三、四章 MMoH 与 HR-MMoH 提供研究空间。

2.1 多模态大语言模型

2.1.1 多模态大语言模型的模型架构

多模态大语言模型 (MLLMs) 将视觉、语言与 (可选) 生成能力统一到以大规模语言模型为核心的异构架构中。其典型组成如图1.1所示，下文结合对各模块的代表性实现做详细展开。

Modality Encoder 即模态编码器，负责将图像、视频、音频等非文本输入编码为高维特征表示，以便与语言模型的词嵌入空间统一。实践中多采用在大规模图文或视频等多模态数据上预训练好的视觉模型作为编码器：CLIP[?] 通过对比学习视觉信息，被 LLaVA[?]、MiniGPT-4[?] 等广泛采用；ViT[?] 及其更大规模的变体 [?] 则将图像切分为多个小的 patch 后经 Transformer 编码，让其它模态信息的处理也使用了与语言模型相同的架构，在 Flamingo[?]、InternVL[?] 等工作中用作视觉 backbone。InternVL[?] 将视觉基础模型规模扩展至 60 亿参数，并在多图、长文本与文档理解上取得领先；Qwen-VL[?] 是阿里巴巴最先进的多模态模型，支持百万像素级高分辨率与多轮对话，在 DocVQA、ChartQA 等基准上表现突出。模态编码器输出通常为序列形式的特征，再经后续模块映射到 LLM 空间，从而完成视觉等其它模态数据与语言模型的对齐的第一步 [?]

Input Projector 将编码器输出的多模态特征映射到与 LLMs 词嵌入维度与语义空间对齐的表示，使 LLMs 能够将视觉等信息与文本统一处理。常见实现包括线性层、轻量 Transformer 层、以及 BLIP-2[?] 提出的 Q-Former (通过可学习的 query 从编码器特征中抽取与任务相关的表示)；InstructBLIP[?] 在此基础上引入指令感知的 Q-Former，根据用户指令从图像中抽取与任务更相关的特征，在 ScienceQA 等零样本任务上显著优于 BLIP-2。该模块参数量远小于 LLM，常与编码器一起在预训练或对齐阶段更新。

LLM Backbone 即大规模语言模型主体，承担主要的语言理解与生成计算，参数量通常占整个 MLLM 的 80%–90%[? ?]。基座模型多选用已在海量文本上预训练好的 Transformer 模型，如 GPT 系列[? ?]、LLaMA[?] 等；在资源受限或强调高效微调时，常对 LLMs 进行冻结训练或仅微调部分层。

Output Projector 与 Modality Generator 用于更加复杂的多模态生成任务（如图生图、文生图、文生视频、文生音频等）：前者将 LLMs 的输出映射到模态生成器可接受的表示空间，后者根据该表示生成图像、视频或音频。生成器多采用扩散模型，如 Stable Diffusion[?] 中基于 U-Net[?] 的潜在扩散结构。Next-GPT[?]、DreamLLM[?] 等工作在此基础上实现了任意模态到任意模态的生成与转换，极大程度上扩展了多模态技术的应用范围。

在训练策略方面，BLIP-2[?] 率先系统性地采用“冻结 Encoder + 冻结 LLM + 仅训练轻量 Projector”的范式，在显著降低训练成本的同时达到与全参数微调可比的效果，后续多数 MLLMs 均沿用或发展此类设计。这种模块异构（编码器、投影层、LLM、生成器在结构、参数量与计算特性上差异显著）与分阶段冻结策略，使得在异构 GPU 集群上为各模块分配合适的算力与显存、并设计均衡的并行与调度策略变得尤为困难。

从实现角度看，编码器多为卷积或 ViT 类结构，单层计算与显存占用与 Transformer 解码层不同；投影层参数量小、计算密集度高，在流水线中若与 LLMs 层混合划分，易造成流水级间负载不均；LLMs 主体层数多、激活体积大，常需配合重计算与张量并行以控制单卡显存。生成器（若存在）多为 U-Net 或扩散结构，其前向与反向计算模式与语言模型差异较大。因此，在设计面向 MLLMs 的并行与调度方案时，需要针对各子模块的特性分别建模计算、显存与通信，并考虑不同训练阶段下冻结比例变化带来的动态负载差异，而不能简单套用单一 Transformer 的划分假设；这也为本文第三章中针对各模块与冻结状态的细粒度划分与调度提供了直接动机。

2.1.2 多模态大语言模型的训练数据

在数据与模型规模的文献共识方面，Kaplan 与 Chinchilla 等关于扩展定律与计算最优配比的讨论[? ? ?] 已经在第一章提及，此处不再复述。大语言模型通常在海量文本语料（如 Common Crawl、The Pile[?]、书籍与代码）上预训练，规模可达数万亿 token。对 MLLMs 训练而言，关键之处在于：在文本预训练之外，还需大规模图片 - 文本或视频 - 文本等多模态配对数据支撑对齐、指令化与生成等训练阶段，使存储与计算压力在单机维度上进一步凸显。多模态理解与推理的评估常采用 MMMU[?] 等多模态评估基准[?]，其题目与数据形态的多样性也反过

来推动训练数据在模态与任务类型上的扩展。

除数据体量本身外，当前 MLLMs 的主流训练配方还往往同时推高有效批量规模与序列长度，从而对单设备显存形成额外压力。一方面，为稳定梯度估计、提高分布式下优化器收敛性，预训练与指令微调在实践中常采用较大的全局批量（global batchsize），并借助梯度累积、数据并行等手段在单步内等效放大 micro-batch，使部分中间激活需常驻或频繁换入换出显存，这部分中间激活随着批量大小增大线性增加；另一方面，文档与图表理解、多图对话、高分辨率输入与视频帧序列等任务，使经 patch 化或采样后的视觉 token 与文本 token 拼接后的序列显著变长，语言侧亦普遍向更长上下文扩展 [?]，代表性工作如 LLaVA-NeXT 等亦持续抬高训练与推理中的上下文规模 [?]。在 Transformer 模型结构下，注意力与中间激活的显存占用随序列长度与批量近似线性或超线性增长，多模态序列还会在长度维上叠加视觉 token，进一步增加中间激活的显存占用。因此，即便模型参数量固定，更大 batchsize 与更长序列仍会与模块异构带来的不均匀显存分布相叠加，使显存成为制约单卡可训练配置与端到端吞吐的关键因素之一，导致重计算技术成为现在训练 MLLMs 的必要技术。这也为本文第四章中针对更大 batchsize 与更长序列的重计算优化提供了直接动机。

表2.1列举了若干代表性数据集的规模与用途。图像方面，Open Images[?] 包含约 900 万张图像及框注，存储需求约 18TB；JFT、LAION 等内部或开放图文对数据集规模可达数亿至数十亿样本，用于视觉—语言预训练。视频方面，YouTube-8M[?] 等基准包含数百万视频与标签，用于视频理解与生成的数据集往往需要 PB 级存储与高吞吐的数据管线。文本方面，Common Crawl、The Pile 等为 LLM 预训练提供数万亿 token 的语料。综合上述规模可知，单机显存既难以容纳完整模型与优化器状态，单机存储与 I/O 带宽也难以在训练周期内高效供给全量数据；在这一双重约束下，并行训练（数据并行、模型并行及其组合）成为必然选择。与此同时，高质量的输入以及对于更长上下文的需求，对计算设备的显存也提出了更高要求，重计算成为不可或缺的技术手段。

表 2.1 多模态大语言模型训练中常用的部分数据集规模示意

类型	规模量级	存储量级	典型用途
图像文本对	数百万 – 数十亿	TB–PB	图文预训练、对齐（如 Open Images[?]）
视频	数百万 – 数亿	百 TB–PB	视频理解、生成（如 YouTube-8M[?]）
文本 / 语料	数百亿 – 数万亿 token	PB 级	LLM 预训练、指令数据

2.1.3 多模态大语言模型的训练过程

多模态大语言模型的训练通常采用多阶段、分模块冻结的策略，在不同阶段设定不同的训练目标与可训练参数集合，以兼顾训练速度与模型质量 [? ?]。各阶段的先后顺序遵循从粗粒度对齐到细粒度能力塑造的递进关系：典型阶段包括（1）模态对齐（或预训练），在大规模图文对或视频—文本对上训练 Input Projector（及可选的部分 Encoder/LLM），使视觉等模态特征与 LLM 语义空间对齐；（2）指令微调（Instruction Tuning），在高质量指令—回答数据上微调，以提升遵循指令与对话能力，此时常冻结 Encoder、仅微调 Projector 与 LLM 部分层；（3）多模态生成微调（若需文生图等），在生成数据上训练 Output Projector 与 Modality Generator，LLM 可冻结或低秩微调。各训练阶段对冻结与更新的模块设定不同，因而在同一模型上，不同阶段对应不同的计算与显存需求分布。换言之，模块异构在时间维度上表现为阶段依赖的、动态变化的负载形态。实践中，各阶段采用的典型超参与数据规模因模型与任务而异：LLaVA[?] 在约 60 万图文对上做视觉—语言对齐与指令微调；InstructBLIP[?] 在 13 个指令化数据集上微调；Qwen-VL[?]、LLaVA-NeXT[?] 等则进一步扩大指令数据与上下文长度；闭源多模态模型如 Gemini[?]、PaLM[?] 系列则在更大规模数据与算力下完成预训练与对齐，其具体数据配比与阶段划分多未完全公开。

冻结会显著改变各模块的反向计算量与显存占用。其原因是：冻结参数不参与梯度计算与更新，因而不需为其保存梯度或中间激活用于反向；优化器状态（如 Adam 的动量与方差）仅需为可训练参数维护，故冻结比例越高，单步显存与计算量通常越低 [?]。在流水线并行下，若某流水级被冻结，其对应设备上的反向计算远少于未冻结的流水级，容易造成流水级间负载不均与流水线气泡，导致流水线效率下降。在异构 GPU 集群下，设备间本就存在显存、算力与带宽差异；不同训练阶段的冻结策略带来的动态模型异构（即同一模型在不同训练阶段呈现不同的有效计算与显存分布）与设备异构叠加，使得静态的、一次性确定的划分与调度难以在各阶段均达到较高的训练效率。因此，一种能够随训练阶段与冻结状态自适应调整划分与调度的方案具有明确价值，也是本文第三章 MMoH 设计的目标之一。

图2.1概括了上述多阶段训练中常见的模块冻结与更新关系，便于理解不同阶段对算力与显存的需求差异。

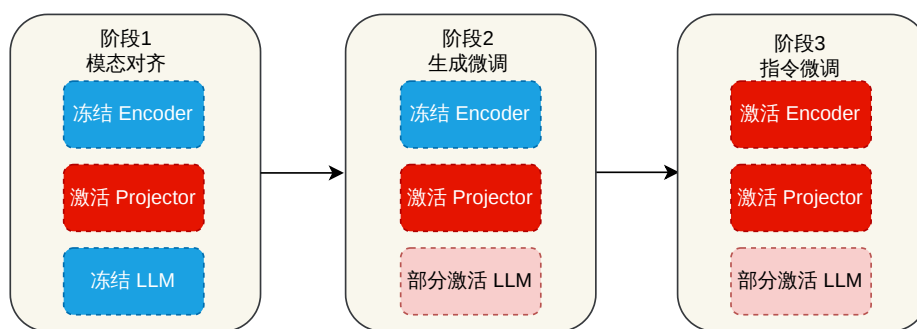


图 2.1 MLLM 多阶段训练中典型模块冻结与更新关系示意

2.2 并行与分布式训练技术

2.2.1 数据并行与模型并行

数据并行（DP）。 数据并行将训练数据在各计算设备间划分，每个设备持有一份完整模型副本，本地前向传播与反向传播后通过梯度 All-Reduce 集合通信同步梯度数据 [? ? ?]。如图2.2所示，各设备独立计算所分配 mini-batch 的梯度，再在前向传播和反向传播都结束后聚合梯度并更新一致参数。DP 实现简单、扩展性好，是 PyTorch DDP[?]、Horovod[?] 等框架的默认方式。PyTorch 的 FSDP[?] 在数据并行基础上对参数、梯度与优化器状态进行分片，与 ZeRO-3 思路相近，可在单卡显存受限时支撑千亿级稠密模型训练；Parallax[?] 针对稀疏与异构场景对参数服务器与 All-Reduce 做了灵活选择。数据并行的局限在于单卡需容纳完整模型与优化器状态（DP）或需额外通信聚合分片（FSDP），无法单独支撑超大参数规模，常与张量 / 流水线并行或 ZeRO 组合使用。

张量并行（TP）与序列并行（SP）。 张量并行在层内划分模型权重与计算，使多设备协同完成单层的前向与反向，组合后等价于完整层 [? ?]。如图2.3所示，每设备仅持有部分参数与激活，层间通过 All-Gather 或者 All-Reduce 等集合通信传递梯度数据。Megatron-LM[?] 针对 Transformer 提出按行 / 列交替划分 FFN 与注意力权重，在减少同步次数的同时控制通信量；GShard[?] 在 MoE 中结合张量并行与专家划分。TP 通信密集且对带宽敏感，实践中多限于单节点或高带宽拓扑内，并与流水线、数据并行组合 [?]。序列并行（SP）针对长序列场景，将输入沿序列维划分到多卡，每卡仅处理一段子序列，通过通信在注意力等层交换信息以保持语义一致 [? ?]。DeepSpeed-Ulysses[?] 将序列均匀分片，在注意力层采用双阶段 All-to-All 重分布，使通信量随序列长度与设备数成比例缩放，支持百万级 token

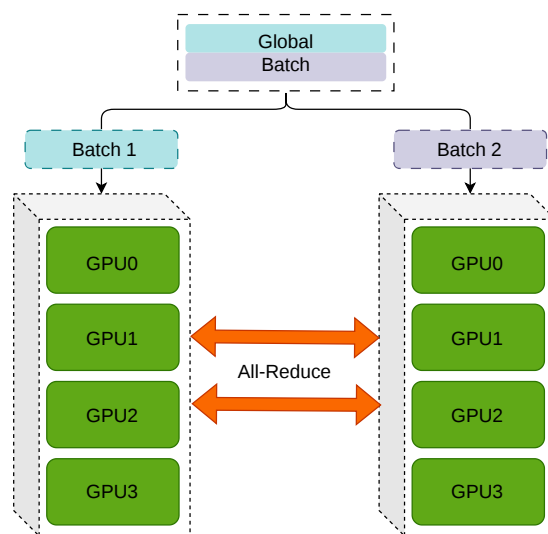


图 2.2 数据并行示意

训练；Megatron 中的序列并行实现常与张量并行共用进程组，与 TP、PP、DP 组合构成多维混合并行 [?]。

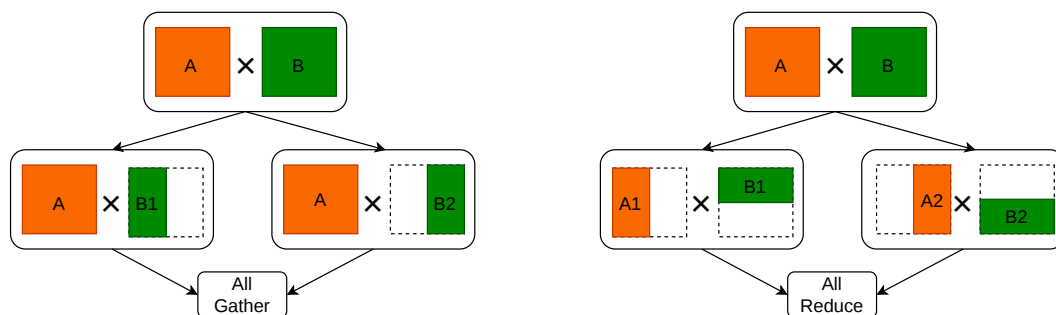


图 2.3 张量并行示意

流水线并行（PP）。 流水线并行在层间划分模型，每设备持有若干连续模型层构成一个流水级，如图2.4所示。GPipe 使用多个 micro-batch 依次流过各流水级以重叠计算、降低流水线气泡率 [?]。PipeDream[?] 提出 1F1B（1 Forward 1 Backward）等异步调度流水线技术；TeraPipe[?] 在序列内做 token 级流水线，实现更细粒度的层间并行。Chimera[?] 采用双向流水线减少气泡；Hanayo[?] 等波式调度进一步优化多流水级下的计算重叠。近年来，ZeroBubble[?] 在同步语义下将反向拆分为“对输入的梯度”与“对参数的梯度”两阶段，结合 1F1B1W 等调度与参数更新重叠，实现了近似零气泡，在相同显存下相较 1F1B 提升吞吐约 23%–31%；DualPipe[?] 采用双向 micro-batch 流动与前后向重叠，在 DeepSeek 等超大规模训

训练中用于降低 bubble 与通信压力。PP 通信量相对较小，适合跨节点扩展，常与节点内 TP、全局 DP 组合 [?]。

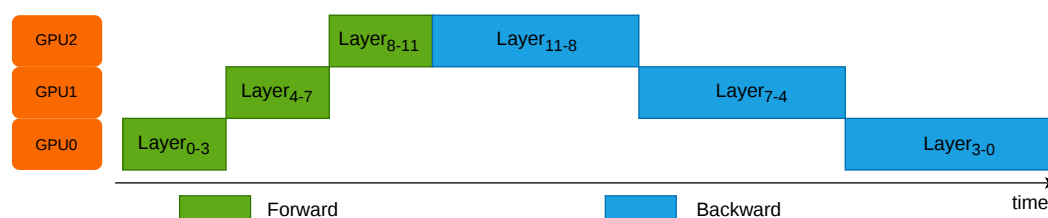


图 2.4 流水线并行示意

上下文并行（CP）与专家并行（EP）。 上下文并行（CP）在如今序列长度越来越长的场景下得到了广泛应用：单卡显存难以容纳全长序列的激活时，将输入沿序列维度划分到多卡，在注意力层通过 All-to-All 集合通信进行分块计算保持注意力分数的完整性。与张量并行共用进程组的序列并行（Sequence Parallelism）不同，上下文并行把模型输入在序列维度做切分，不同的序列块使用不同的 GPU 进行计算，称为 Context Parallelism（CP）[?]。CP 在 8K+ token 场景下显存收益显著，但会引入额外通信与拓扑约束，需与 Flash Attention、重计算等技术配合使用。专家并行（EP）针对混合专家模型（MoE）：不同 token 由不同子网络（专家）处理，EP 将多个专家分布到不同设备，每设备仅持有部分专家，前向时根据路由将激活发送至对应专家所在设备，再汇总结果 [?]。与张量并行（每设备持所有专家的部分权重）不同，EP 下每设备只存部分专家的全量权重，通信模式为 All-to-All 的 token 重分布，适合专家数多、单专家规模适中的 MoE。GShard[?] 将 MoE 与张量并行结合并做自动分片；DeepSeek-MoE[?] 等在有限资源下通过 MoE 扩展语言模型规模；DualPipe[?] 等也在流水线中结合专家并行以支撑超大规模 MoE 训练。EP 的主要挑战是专家负载不均与路由带来的通信开销，需高带宽互联与负载均衡设计。图2.5展示了专家并行中的典型路由过程：门控网络先为 token 选择 Top- k 专家，再通过 All-to-All 集合通信将 token 分发到对应 GPU 上的专家执行计算，最后汇聚计算结果。

2.2.2 零冗余优化器（ZeRO）

ZeRO[?] 在数据并行基础上，将优化器状态、梯度与参数分片存于各计算设备，前向与反向时按需聚合，在保持 DP 通信量的前提下显著降低单卡显存，可以支持上千亿参数模型的训练。如图2.6所示，ZeRO 三阶段分别将优化器状态、梯度与参数进行分片来减少单卡的显存占用；ZeRO-Infinity[?] 进一步将部分状态卸载至 CPU/NVMe 以突破 GPU 显存墙。ZeRO++[?] 通过量化与通信重算减少跨节

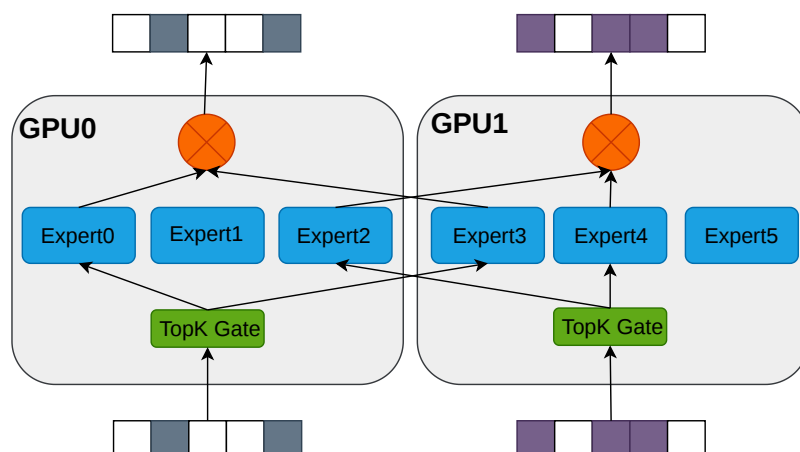


图 2.5 专家并行（EP）中门控路由与专家分布示意

点通信量；MiCS[?]、AMSP[?] 等在云环境与统一划分空间上做了扩展。ZeRO-3 与张量并行、流水线并行的组合在工程上存在约束（如 DeepSpeed 与 Megatron 的 3D 并行通常采用 ZeRO-1/2 与 TP+PP），需根据框架与集群拓扑谨慎配置[?]，在实际部署中需在显存节省、通信开销与实现复杂度之间权衡。ZeRO 与 TP、PP、SP 等组合构成当前大模型与 MLLM 训练的主流技术手段。

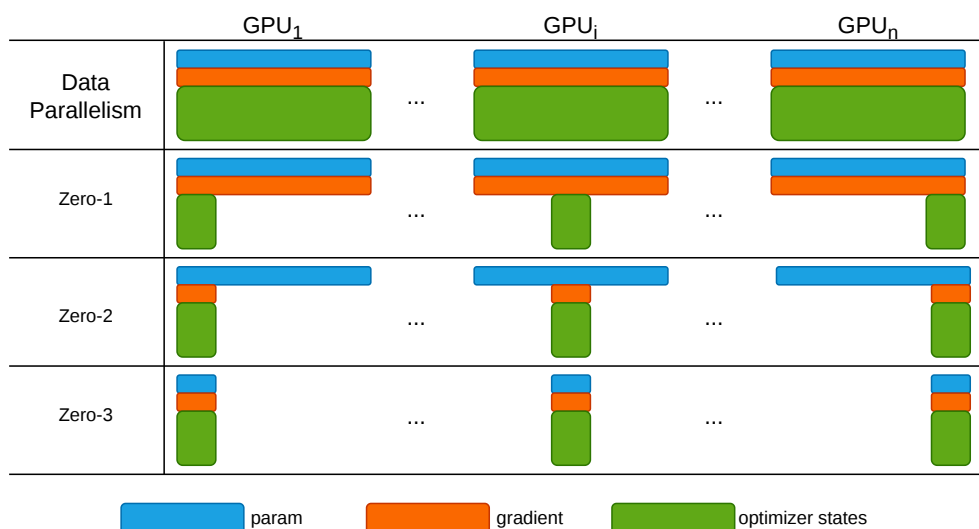


图 2.6 零冗余优化器 ZeRO 示意

2.3 异构集群上的大模型并行训练

第一章从实际部署出发，已说明常见情形：同一集群中往往混有多代 GPU，显存容量、算力与互联带宽并不一致；若仍按同构假设选用默认并行配置，容易

出现负载倾斜、通信受限以及单点设备制约整体效率等问题。本节不再展开现象层面论述，而沿文献脉络梳理近年面向算力与互联不均的异构集群并行训练工作，并说明其在多模态大模型——编码器、投影层与语言骨干结构差异显著，且训练各阶段冻结范围常需调整——的情形下仍存在的不足。

2.3.1 异构 GPU 集群下的并行训练策略

围绕异构环境下的并行训练，已有若干代表性工作从不同角度展开了探索。

HetPipe [?] 较早提出了针对异构 GPU 设备的解决思路。它将若干 GPU 组织为虚拟工作节点（VW），节点内部以流水线并行处理小批量，节点之间叠加数据并行来提升训练吞吐。通过强弱设备搭配成组，该方法在一定程度上避免了算力较弱的设备成为单一流水瓶颈。不过，它并未对 VW 内部的拓扑结构、设备排列与通信开销进行精细建模。更关键的是，该方法默认模型为相对规整的层堆叠结构，而对于编码器、投影层与大语言模型各自“形态迥异”的多模态大语言模型，以及训练过程中分阶段切换冻结策略等复杂场景，并没有现成的解法。

Whale [?] 的思路则相对直观：既然设备显存与算力天然存在差异，不如按设备能力为其分配差异化的批次大小与分片策略，在数据并行与张量并行的维度上将负载“摊平”。这一方案将“大卡多算、小卡少算”的工程直觉转化为可操作的实施路径。但它的划分粒度止步于子图层级，未能与 ZeRO 等显存优化手段打通。当集群规模扩大或异构维度增多时，单纯依赖比例切分往往难以兼顾通信效率与显存安全。

AMP [?] 则将并行策略的搜索推向了自动化方向。它将层间结构差异抽象为“模型异构”，将设备显存、算力与链路特征抽象为“设备异构”，在建模与量化的基础上，于 DP、TP、PP 的组合空间中建立开销模型，并借助动态规划求取近似最优配置。实验结果显示，在异构场景下该方法较基线有明显收益。但问题在于，全局划分的粒度仍然较粗，通信与显存的估计误差会直接传导至最终配置方案；若缺乏显存上界等硬性约束，实际部署中仍可能发生显存溢出（OOM），通常还需配合激活重算与分片预算等手段进行额外调参，才能稳定运行。

上述工作在缓解负载不均衡与显存压力方面提供了有益借鉴，但这些方法都有一个共同前提：模型主体可近似为结构相近的 Transformer 层堆叠。当研究对象转向模块边界清晰、各模块计算与显存曲线差异显著的多模态大语言模型，并需应对训练过程中分阶段冻结策略的动态变化时，原有的共同前提被打破，方法的有效性也随之下降。

2.3.2 异构 GPU 集群并行训练的工业实践

近年来，工业界研究更关注如下问题：在模型与数据规模确定的前提下，异构集群应如何选配 GPU 数量与型号、流水级如何映射到设备，以及如何在成本与时延约束下取得可接受的训练性能。**Espresso**[?] 针对上述需求构建框架：**Profiler** 在各类 GPU 上测量迭代时间与显存占用，**Allocator** 完成设备分配，**Stage Placer** 确定流水级映射，**Performance Estimator** 估计给定配置下的吞吐。用户输入模型、数据集、训练轮次与 SLO 等指标后，系统输出资源与 stage 方案，在成本与效率之间折中。它将测量、资源分配与性能估计串成闭环，但缺乏与 MLLM 多模块协同等方面的深度结合，实际用来训练 MLLMs 时仍然不能充分利用异构设备。

多厂商设备的混合则进一步放大了上述需求。**HetHub**[?] 针对华为昇腾、AMD、NVIDIA 等卡混在一起的集群，做了统一通信、性能预测和自动并行规划；在 768 张异构卡上训练 Llama-140B，可达到理论性能上界约 97.49%，表明多厂商混部集群同样能够支撑超大模型训练，但对系统工程要求较高。**HexiScale**[?] 则把 DP、PP、TP 的非对称划分写成约束优化，用层次图划分给各卡分计算，在 7B–30B 规模上 MFU 与同构系统只差大约 3.5%，相当于给“按能力不对称切分”补了一套可复用的算法。

Zorse[?] 更贴近大规模 LLM 训练中的常见矛盾：流水线与数据并行如何组合以降低通信与显存开销；流水级能否非对称划分、同一数据并行组内能否容纳能力不同的设备。它将上述灵活性统一纳入实现，并配备自动规划器，实验表明相对既有方法可获得一定训练加速。

ZeRO 系方法在异构环境中亦不能完全照搬同构假设下的经验。原版 ZeRO[?] 中参数与梯度的 All-Gather 等集合通信，在跨节点且链路带宽参差不齐时易成为瓶颈；MiCS[?] 在超大模型与复杂网络拓扑下对高带宽链路的利用仍不充分；ZeRO++[?] 借助量化与通信重算降低流量，需在收益与精度、通用性之间权衡；AMSP[?] 等以削减跨节点通信为主，对单机内显存、算力与带宽细粒度错配的综合利用讨论相对较少。

综合上述工作可见，学术界与工业界正通过自动规划、测量驱动的配置与约束优化等途径，降低对手工调参的依赖。然而，这些路线多数仍以通用 LLM 或“集群近似同构后再谈扩展”为基本前提；当训练对象变为模块边界清晰、各模块资源曲线差异显著的多模态大语言模型，且需在多阶段训练中反复调整冻结范围时，计算图随阶段明显变化，现有方法尚难与之对齐并形成系统化的专门方案。若将上述结论与本章关于并行划分及异构集群扩展的综述一并考虑，在 MLLM 的模块异构与分阶段冻结与显存、算力、通信三重错配同时出现的条件下，仍缺少将划分、调度与显存策略统筹考虑的成套联合优化。

工程落地层面，还需应对异构环境下的性能预测精度、与 Megatron、DeepSpeed 等框架的集成方式，以及多阶段训练中划分与调度能否随阶段演化而及时更新等问题。Frenzy[?] 等面向内存感知的无服务器调度研究，可为资源与阶段放置提供借鉴；Espresso、Zorse 等则尝试将性能分析、资源配置与并行规划封装为较完整的工具链。即便如此，面向 MLLM 的模块级划分以及冻结策略切换时的自适应调度，仍未形成成熟的一体化方案，本文提出的 MMoH 即针对上述缺口展开研究与系统设计。

2.4 重计算显存优化技术

大规模模型与长序列训练中，前向传播产生的中间激活（activation）需在反向传播时用于梯度计算，若全部保存则显存占用随层数与序列长度线性甚至平方增长，成为训练瓶颈。重计算（Recomputation），又称激活检查点 (Activation Checkpointing)，通过“以计算换显存”的思想缓解上述压力，是后续各小节讨论的基础。本节依次从策略分类与训练框架中的实现及其局限两方面展开综述。

2.4.1 重计算基本原理与分类

重计算具体指：前向传播时仅保存部分激活或仅保存检查点处的输入，反向传播时对未保存的区域重新执行前向传播以得到所需激活值，如图2.7所示，从而在显存与计算之间进行权衡，是缓解显存压力的常用手段 [?]。根据检查点放置的粒度和策略，可大致分为全部重计算、选择重计算与细粒度重计算三类；以下分别简述其思路与适用场景。

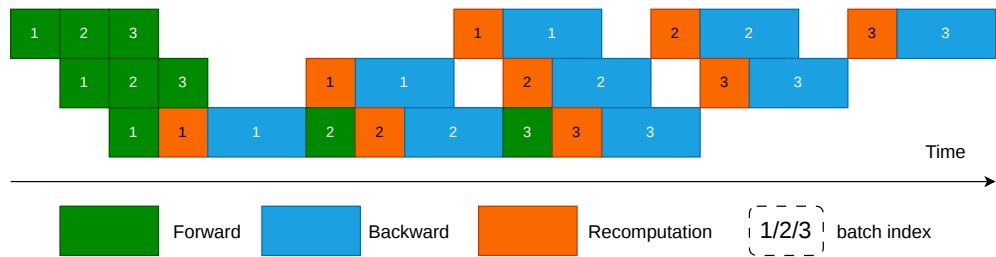


图 2.7 重计算（激活检查点）示意

全部重计算。 全部重计算（full recomputation）策略在前向阶段不保存任何中间激活，仅在若干“检查点”层保存该层的输入（或仅保存网络输入），反向传播需要某段中间激活时，从最近的检查点重新执行前向至该位置再计算梯度。极端情况下仅保存网络输入，则显存占用可降至与深度无关的常数级别，但反向传播需对整网做一次完整前向重算，计算开销约增加前向传播的一倍。Chen 等 [?] 针对

链式计算图证明了在仅增加一次额外前向的前提下，通过动态规划可求得检查点的最优放置，使显存从 $O(n)$ 降为 $O(\sqrt{n})$ (n 为层数)，并在千层残差网络等实验中实现了显存的大幅下降（如 48 GB $\rightarrow$ 7 GB）与约 30% 的训练时间增加。全部重计算实现简单、显存收益最大，适合显存极度紧张、算力相对充裕的场景，但计算冗余较高。

选择重计算。 选择重计算（selective recomputation）不是对整个 Transformer 层统一做 checkpoint，而是只对显存占用大但重算代价相对较低的部分子模块执行重算，未选部分仍正常保存激活。Korthikanti 等 [?] 在大规模 Transformer 中提出的 selective activation recomputation 即属于此类：其核心思想是优先重算注意力中的 QK^T 、softmax、dropout 以及与 V 的乘法等中间激活，因为这些张量随序列长度和注意力头数增长较快，但额外 FLOPs 开销相对有限。相较整层重计算，该策略的显存收益略小，但能显著降低重算开销；论文报告其总时间开销约为 7%，明显低于整层重算的 39%[?]。在工程实现上，Megatron-LM 与 NeMo 将其落地为 selective 粒度，默认仅重算 core_attn，也可扩展到 mlp、moe、layernorm 等指定子模块 [??]。

细粒度重计算。 细粒度重计算（fine-grained recomputation）在算子或更小粒度上决定哪些激活保存、哪些重算，从而更精细地利用不同算子的计算—显存特性。与上一段“选择重计算”主要回答“哪些模块需要重算”不同，细粒度重计算更关注“如何在模块内部重算”。例如，Transformer 中可不把整个注意力或 MLP 作为单一单位处理，而是进一步细化到 LayerNorm、dropout、MoE 激活函数、或某些投影输出等中间张量：前向时仅保留最少输入或元数据，反向时再局部恢复需要的结果。Megatron-LM/NeMo 中的 output-discarding checkpoint 就体现了这种思路：它并非简单地把整段前向再执行一遍，而是在前向后主动丢弃部分输出张量的存储，在反向阶段借助 hook 触发局部重算，从而进一步压缩激活占用 [??]。这类方法比“按层”或“按子模块”的重计算更贴近实现细节，能够利用不同算子的显存占用、通信需求与 FLOPs 差异做更细致的权衡。Lynx[?] 进一步将激活重算与通信重叠调度，在 1.3B–23B GPT 模型上相较现有方法取得最高约 $1.37\times$ 的加速，表明细粒度重算若与通信、并行调度协同设计，还可以同时改善显存效率与训练吞吐。细粒度策略可结合模型结构（如 MLLMs 中 Encoder、Projector、LLM 各模块的算子组成）与异构设备的显存、算力差异，在不同设备或不同 stage 上采用不同的重计算粒度，是面向异构集群与 MLLM 的潜在优化方向。

2.4.2 重计算的主流实现及其局限性

在 Megatron-LM 与 NeMo 等框架中，重计算被实现为可配置的多级粒度。全部重计算（full）以整个 Transformer 层为粒度：前向阶段仅仅保存每层输入，反向时重算整层前向，显存下降显著但单层计算开销增加 30% 以上。框架进一步提供两种全层模式：*block* 模式通过参数指定每个流水线 stage 内参与重计算的层数，可仅对部分层做重计算以在显存与计算间折中；*uniform* 模式以连续多层的“块”为重计算单位，块内仅存块首输入，适用于层数很多或层间激活相对较小的模型，可节省激活显存但会增加重算缓冲。选择性重计算（selective）则仅在子模块级别重算：例如仅对自注意力块做重计算（保存注意力块输入，重算 softmax、dropout、QKV 等中间激活），而 MLP 等不重算。自注意力激活随序列长度平方增长、占显存大头，但其重算成本相对线性投影层较低，因此选择性重算能以较小计算代价获得较高显存收益；用户还可指定 `core_attn`、`mlp`、`moe` 等模块是否参与重算。在使用 Flash Attention 时，框架通常强制对注意力做选择性重算以配合其显存布局。上述多级粒度（全层 *block/uniform*、子模块 *selective*）为实际训练中按显存预算与算力调节重计算策略提供了细粒度控制，与 Korthikanti 等 [?] 的序列并行与选择性激活重算相结合，已广泛应用于大规模 Transformer 与 LLM 训练。

尽管如此，现有重计算策略多面向同构设备设计：重计算层数、块大小、参与重算的子模块等均需用户或策略脚本手动指定，且通常假设各设备显存与算力一致，无法根据异构集群中不同设备的显存余量与算力差异自动调节（例如在显存紧张设备上加大重算比例、在算力瓶颈设备上减少重算）。将重计算与流水线并行、stage 划分及异构分配结合，可进一步缓解显存瓶颈并提升整体吞吐：例如在流水线各 stage 内独立设置重计算策略，或在显存紧张的设备上采用更激进的重计算、在算力紧张处适当多存以减少重算，但上述调节目目前仍依赖人工或离线调参。Megatron-LM、NeMo 等支持按层或子模块（如 `core_attn`、`mlp`）配置选择性重算 [?]；Colossal-Auto[?] 等将自动并行策略搜索与重计算统一自动化，为大规模模型与异构环境下的联合优化提供了参考；在异构 GPU 集群上，针对 MLLM 的模块异构与分阶段冻结，设计能自动适应设备异构的细粒度重计算策略是本文的工作方向之一。

2.5 本章小结

本章承接第一章的论述，从文献角度分四方面综述了 MLLM 架构与训练数据 / 范式、同构假设下的并行与 ZeRO 基础设施、异构 GPU 集群上的系统化工作，以及重计算策略与框架实现。首先介绍了 MLLMs 的典型架构、训练数据规模与训练目标，以及模块异构与分阶段冻结带来的计算—显存需求差异。随后综述了数

据并行、张量并行、流水线并行、序列并行与专家并行等多维混合并行方式，以及 ZeRO 系列显存分片优化，指出其同构假设下的广泛应用与在异构、模块异构场景下的局限。接着在异构 GPU 集群一节中分两部分：先概述显存、算力与通信三重异构及 Whale 等并行与自动搜索路线，再讨论 Espresso 等系统化资源规划框架及 ZeRO 类显存分片在异构环境下的局限，并指出其对 MLLMs 模块异构与分阶段冻结的适配仍不足。最后在重计算一节中分两部分展开：先概述重计算思想及全层、选择与细粒度三类策略；再归纳现有框架中的实现。综合来看，MLLM 的模块异构与分阶段冻结、以及异构集群的显存—算力—通信差异，共同构成了现有同构导向并行与统一重计算策略难以直接应对的挑战。上述分析为本文第三章的 MMoH 混合并行框架与第四章的 HR-MMoH 异构重计算提供了问题背景与动机。

致 谢

作者在学期间取得的学术成果

发表的学术论文

[1] J. Fan et al., "MMoH: Efficient Training of Multimodal Large Language Models on Heterogeneous Clusters," 2025 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Shenyang, China, 2025, pp. 990-997, doi: 10.1109/ISPA67752.2025.00135.(CCF-C 类会议 第一作者)

附录 A 本文中英文对照表

表 A.1 给出正文常用术语的中英文对照；第二列在必要时附缩写或原文记号。

表 A.1 本文主要术语中英文对照表

中文	英文
自注意力机制	Self-Attention
深度神经网络	Deep Neural Network (DNN)
卷积神经网络	Convolutional Neural Network (CNN)
循环神经网络	Recurrent Neural Network (RNN)
计算机视觉	Computer Vision (CV)
自然语言处理	Natural Language Processing (NLP)
语音识别	Speech Recognition (SR)
大语言模型	Large Language Model (LLM)
多模态大语言模型	Multimodal Large Language Model (MLLM)
通用人工智能	Artificial General Intelligence (AGI)
迁移学习	Transfer Learning
少样本学习	Few-Shot Learning
零样本学习	Zero-Shot Learning
扩展定律	Scaling Laws
图文对比学习 CLIP	Contrastive Language-Image Pre-Training
最优先进水平	state-of-the-art (SOTA)
模态编码器	Modality Encoder
输入投影层	Input Projector
LLM 基座	LLM Backbone
输出投影层	Output Projector
模态生成器	Modality Generator
扩散模型	diffusion model
模块异构	module heterogeneity
模态对齐	modality alignment
指令微调	instruction tuning
数据并行	data parallelism (DP)

续下页

续表 A.1

中文	英文
分布式数据并行	Distributed Data Parallel (DDP)
张量并行	tensor parallelism (TP)
流水线并行	pipeline parallelism (PP)
序列并行	sequence parallelism (SP)
上下文并行	context parallelism (CP)
专家并行	expert parallelism (EP)
混合专家	Mixture of Experts (MoE)
混合并行	hybrid parallelism
微批次	micro-batch
小批量	mini-batch
一前向一反向调度	One-Forward-One-Backward (1F1B)
全归约	AllReduce
全收集	All-Gather
归约分散	Reduce-Scatter
全对全通信	All-to-All
点对点通信	Peer-to-Peer (P2P)
参数服务器	parameter server
流水级	pipeline stage
流水线气泡	pipeline bubble
混合精度训练	mixed-precision training
全分片数据并行	Fully Sharded Data Parallel (FSDP)
零冗余优化器	Zero Redundancy Optimizer (ZeRO)
环形注意力	Ring Attention
激活 / 激活值	activation
显存溢出	Out Of Memory (OOM)
负载均衡	load balancing
剖析 / 性能剖析	profiling
设备拓扑	device topology
整数规划	integer programming
复合粒度（模型抽象）	multi-granularity model abstraction

续下页

续表 A.1

中文	英文
成本模型	cost model
服务等级目标	Service Level Objective (SLO)
模型算力利用率	Model FLOPs Utilization (MFU)
虚拟工作节点	Virtual Worker (VW)
剖析器	Profiler
GPU 分配器	GPU Allocator
阶段放置器	Stage Placer
全部重计算	full recomputation
选择重计算	selective recomputation
细粒度重计算	fine-grained recomputation
Top- k 专家路由	Top- k (expert routing)